Міністерство освіти і науки України

Національний технічний університет України

“Київський політехнічний інститут імені Ігоря Сікорського”

Факультет інформатики та обчислювальної техніки

Кафедра інформаційні систем та технологій

Лабораторна робота № 6

із дисципліни «Розробка мобільних застосувань під Android»

Тема: «Дослідження роботи з мережевими запитами (REST API) та асинхронним програмуванням при розробці клієнт-серверного застосунку»

Виконав

Студент групи ІА-31:

Губар Б. О.

Перевірив:

Орленко С. П.

Київ 2026

1. **Мета:**
змоделювати головну проблему наступного року навчання (у вигляді вибору теми дипломного проекту) шляхом самостійного формування завдання на 6 лабораторну роботу.
2. **Завдання:**

Написати програму під платформу Андроїд, яка буде заснована на темі, яка не розглядалась на попередніх лабораторних роботах (попередні навички теж можна використовувати, але «основою» має бути саме нова обрана тема).

Примітка: Очевидно, що конкретного варіанту не передбачено, студент сам формує завдання та вигляд програми. Наприклад, на попередніх лабах не була приділена увага web-у, створенню віджетів, роботі з зовнішнім обладнанням, створенню ігрових застосунків і т.д.

Короткий опис проєкту (Фокус на технологіях)

Мета розробленого застосунку: Створення мобільного клієнта для пошуку, фільтрації та бронювання залізничних квитків, а також адміністрування маршрутів. Основний акцент зроблено на реалізації взаємодії з віддаленим сервером, що не розглядалося в попередніх лабораторних роботах.

Використаний технологічний стек:

- Мережева взаємодія (REST API): Використано бібліотеку Retrofit у зв'язці з конвертером Gson для виконання HTTP-запитів (GET, POST, PUT, DELETE) до зовнішньої бази даних (MockAPI) та парсингу JSON-відповідей у Kotlin дата-класи (TrainRoute).
- Асинхронне програмування: Для уникнення блокування головного потоку (UI Thread) під час мережевих запитів застосовано Kotlin Coroutines (CoroutineScope, Dispatchers.IO, Dispatchers.Main).
- Динамічні списки: Для ефективного виведення великих масивів даних (розкладів потягів та сітки місць у вагоні) імплементовано RecyclerView з власними адаптерами (TrainAdapter, SeatAdapter) та GridLayoutManager.
- Інтерфейс користувача (UI): Дизайн побудовано за принципами Material Design. Використано компоненти TextInputLayout (OutlinedBox), MaterialCardView, MaterialButton та системний DatePickerDialog для вибору дати.

3. **Хід роботи**

RetrofitClient.kt Лістинг:

```
package com.example.trainschedule

import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

object RetrofitClient {
```

```
    private const val BASE_URL = "https://69ea964f15c7e2d51269e9c0.mockapi.io/"

    val apiService: TrainApiService by lazy {
        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .build()
            .create(TrainApiService::class.java)
    }
}
```

TrainApiService.kt Лістинг:

```
package com.example.trainschedule

import retrofit2.http.Body
import retrofit2.http.DELETE
import retrofit2.http.GET
import retrofit2.http.POST
import retrofit2.http.Path
import retrofit2.http.PUT

interface TrainApiService {
    @GET("trains")
    suspend fun getTrains(): List<TrainRoute>

    @POST("trains")
    suspend fun addTrain(@Body train: TrainRoute): TrainRoute

    @DELETE("trains/{id}")
    suspend fun deleteTrain(@Path("id") id: String)

    @PUT("trains/{id}")
    suspend fun updateTrain(@Path("id") id: String, @Body train: TrainRoute): TrainRoute
}
```

TrainRoute.kt Лістинг:

```
package com.example.trainschedule

import com.google.gson.annotations.SerializedName

data class TrainRoute(
    @SerializedName("id") val id: String? = null,
    @SerializedName("tripId") val tripId: Int,
    @SerializedName("trainNumber") val trainNumber: String,
    @SerializedName("startStation") val startStation: String,
    @SerializedName("endStation") val endStation: String,
    @SerializedName("departureTime") val departureTime: String,
    @SerializedName("price") val price: Double,
    @SerializedName("trainType") val trainType: String,
    @SerializedName("carriageCount") val carriageCount: Int?,
    @SerializedName("departureDate") val departureDate: String?,
    @SerializedName("status") val status: String?,
    @SerializedName("bookedSeats") val bookedSeats: String?
)
```

MainActivity.kt Лістинг:

```
package com.example.trainschedule

import android.os.Bundle
import android.util.Log
import android.view.View
import android.widget.ArrayAdapter
import android.widget.Button
import android.widget.EditText
import android.widget.ImageButton
import android.widget.Spinner
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.recyclerview.widget.GridLayoutManager
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext

class MainActivity : AppCompatActivity() {

    private lateinit var recyclerView: RecyclerView
    private lateinit var trainAdapter: TrainAdapter

    private lateinit var etDeparture: EditText
    private lateinit var etArrival: EditText
    private lateinit var btnSearch: Button
    private lateinit var btnAdminLogin: Button
    private lateinit var fabAddRoute: View
    private lateinit var btnSwap: ImageButton
    private lateinit var btnSortPrice: Button
    private lateinit var btnSortTime: Button
    private lateinit var btnSelectDate: Button

    private var allTrains: List<TrainRoute> = emptyList()

    private var isAdminLoggedIn = false
    private var isPriceAsc = true
    private var isTimeAsc = true
    private var selectedSearchDate: String = ""

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        recyclerView = findViewById(R.id.recyclerView)
        etDeparture = findViewById(R.id.etDeparture)
        etArrival = findViewById(R.id.etArrival)
        btnSearch = findViewById(R.id.btnSearch)
        btnAdminLogin = findViewById(R.id.btnAdminLogin)
        fabAddRoute = findViewById(R.id.fabAddRoute)
        btnSwap = findViewById(R.id.btnSwap)
        btnSortPrice = findViewById(R.id.btnSortPrice)
        btnSortTime = findViewById(R.id.btnSortTime)
        btnSelectDate = findViewById(R.id.btnSelectDate)

        recyclerView.layoutManager = LinearLayoutManager(this)

        trainAdapter = TrainAdapter(emptyList(), false,
            { selectedTrain -> showTrainDetailsDialog(selectedTrain) },
            { trainToDelete -> deleteRoute(trainToDelete) },
            { trainToEdit -> showEditRouteDialog(trainToEdit) }
```

```
        )
        recyclerView.adapter = trainAdapter

        btnSelectDate.setOnClickListener {
            val calendar = java.util.Calendar.getInstance()
            val year = calendar.get(java.util.Calendar.YEAR)
            val month = calendar.get(java.util.Calendar.MONTH)
            val day = calendar.get(java.util.Calendar.DAY_OF_MONTH)

            android.app.DatePickerDialog(this, { _, selectedYear, selectedMonth, selectedDay
->
                selectedSearchDate = String.format("%02d.%02d.%04d", selectedDay,
selectedMonth + 1, selectedYear)
                btnSelectDate.text = selectedSearchDate

                btnSelectDate.setBackgroundColor(android.graphics.Color.parseColor("#EDE7F6"))
                btnSelectDate.setTextColor(android.graphics.Color.parseColor("#5E35B1"))
                (btnSelectDate as com.google.android.material.button.MaterialButton).apply {
                    strokeWidth = 0
                    iconTint =
android.content.res.ColorStateList.valueOf(android.graphics.Color.parseColor("#5E35B1"))
                }
            }, year, month, day).show()
        }

        btnSearch.setOnClickListener {
            val dep = etDeparture.text.toString().trim()
            val arr = etArrival.text.toString().trim()

            if (dep.isEmpty() || arr.isEmpty() || selectedSearchDate.isEmpty()) {
                Toast.makeText(this, "Введіть станції та оберіть дату!",
Toast.LENGTH_SHORT).show()
            } else {
                searchTrains(dep, arr, selectedSearchDate)
            }
        }

        btnAdminLogin.setOnClickListener {
            if (isAdminLoggedIn) {
                isAdminLoggedIn = false
                btnAdminLogin.text = "ВХІД ДЛЯ ПЕРСОНАЛУ"
                btnAdminLogin.setTextColor(android.graphics.Color.parseColor("#8E8E93"))
                etDeparture.text.clear()
                etArrival.text.clear()
                btnSelectDate.text = "ОБРАТИ ДАТУ"
                selectedSearchDate = ""
                trainAdapter.updateData(emptyList())
                fabAddRoute.visibility = View.GONE
                trainAdapter.isStaffMode = false
                trainAdapter.notifyDataSetChanged()
                Toast.makeText(this, "Вихід виконано. Активовано режим пасажира.",
Toast.LENGTH_SHORT).show()
            } else {
                showAdminLoginDialog()
            }
        }

        fabAddRoute.setOnClickListener {
            showAddRouteDialog()
        }

        btnSwap.setOnClickListener {
            val from = etDeparture.text.toString()
            val to = etArrival.text.toString()
            etDeparture.setText(to)
            etArrival.setText(from)
```

```
        }

        btnSortPrice.setOnClickListener {
            val currentList = trainAdapter.trains
            if (currentList.isEmpty()) return@setOnClickListener

            val sortedList = if (isPriceAsc) {
                currentList.sortedBy { it.price }
            } else {
                currentList.sortedByDescending { it.price }
            }

            trainAdapter.updateData(sortedList)
            isPriceAsc = !isPriceAsc
            Toast.makeText(this, if (isPriceAsc) "Від найдешевшого" else "Від найдорожчого",
Toast.LENGTH_SHORT).show()
        }

        btnSortTime.setOnClickListener {
            val currentList = trainAdapter.trains
            if (currentList.isEmpty()) return@setOnClickListener

            val sortedList = if (isTimeAsc) {
                currentList.sortedBy { it.departureTime }
            } else {
                currentList.sortedByDescending { it.departureTime }
            }

            trainAdapter.updateData(sortedList)
            isTimeAsc = !isTimeAsc
            Toast.makeText(this, if (isTimeAsc) "Спочатку ранні" else "Спочатку пізні",
Toast.LENGTH_SHORT).show()
        }
    }

    private fun searchTrains(departure: String, arrival: String, date: String) {
        CoroutineScope(Dispatchers.IO).launch {
            try {
                val allTrains = RetrofitClient.apiService.getTrains()
                this@MainActivity.allTrains = allTrains

                val filteredList = allTrains.filter { train ->
                    train.startStation.equals(departure, ignoreCase = true) &&
                            train.endStation.equals(arrival, ignoreCase = true) &&
                            train.departureDate == date
                }

                withContext(Dispatchers.Main) {
                    if (filteredList.isEmpty()) {
                        Toast.makeText(this@MainActivity, "Потягів на $date не знайдено",
Toast.LENGTH_LONG).show()
                        trainAdapter.updateData(emptyList())
                    } else {
                        trainAdapter.updateData(filteredList)
                    }
                }
            } catch (e: Exception) {
                withContext(Dispatchers.Main) {
                    Log.e("TrainSearch", "Помилка завантаження: ${e.message}")
                    Toast.makeText(this@MainActivity, "Помилка зв'язку з базою даних",
Toast.LENGTH_SHORT).show()
                }
            }
        }
    }
```

```
    private fun showTrainDetailsDialog(train: TrainRoute) {
        val dialogView = layoutInflater.inflate(R.layout.dialog_seat_selection, null)
        val spinnerWagon = dialogView.findViewById<Spinner>(R.id.spinnerWagon)
        val rvSeats = dialogView.findViewById<RecyclerView>(R.id.rvSeats)
        val btnBuyTicket = dialogView.findViewById<Button>(R.id.btnBuyTicket)

        val carriageCount = train.carriageCount ?: 10
        val wagons = (1..carriageCount).map { "Вагон №$it" }
        val spinnerAdapter = ArrayAdapter(this, android.R.layout.simple_spinner_dropdown_item,
wagons)
        spinnerWagon.adapter = spinnerAdapter

        rvSeats.layoutManager = GridLayoutManager(this, 4)
        var currentSelectedSeat: Seat? = null

        val seatAdapter = SeatAdapter(emptyList()) { seat ->
            currentSelectedSeat = seat
            if (seat != null) {
                btnBuyTicket.isEnabled = true
                btnBuyTicket.text = "ОПЛАТИТИ ${train.price} ₴"
            } else {
                btnBuyTicket.isEnabled = false
                btnBuyTicket.text = "ОПЛАТИТИ"
            }
        }
        rvSeats.adapter = seatAdapter

        val currentBookedSeats = train.bookedSeats ?: ""

        spinnerWagon.onItemSelectedListener = object :
android.widget.AdapterView.OnItemSelectedListener {
            override fun onItemSelected(parent: android.widget.AdapterView<*>?, view: View?,
position: Int, id: Long) {
                val currentWagonNumber = position + 1

                val generatedSeats = (1..36).map { seatNum ->
                    val seatKey = "$currentWagonNumber-$seatNum"
                    val status = if (currentBookedSeats.contains(seatKey)) SeatStatus.OCCUPIED
else SeatStatus.FREE
                    Seat(seatNum, status)
                }

                seatAdapter.updateSeats(generatedSeats)
                btnBuyTicket.isEnabled = false
                btnBuyTicket.text = "ОПЛАТИТИ"
            }
            override fun onNothingSelected(parent: android.widget.AdapterView<*>?) {}
        }

        val dialog = androidx.appcompat.app.AlertDialog.Builder(this)
            .setTitle("Рейс ${train.trainNumber}: ${train.startStation} →
${train.endStation}")
            .setView(dialogView)
            .setNegativeButton("Закрити", null)
            .show()

        btnBuyTicket.setOnClickListener {
            val selectedWagonNum = spinnerWagon.selectedItemPosition + 1
            val seatNum = currentSelectedSeat?.number
            val newSeatKey = "$selectedWagonNum-$seatNum"

            val updatedBookedSeats = if (currentBookedSeats.isEmpty()) {
                newSeatKey
            } else {
                "$currentBookedSeats,$newSeatKey"
            }
```

```
            val updatedTrain = train.copy(bookedSeats = updatedBookedSeats)

            btnBuyTicket.isEnabled = false
            btnBuyTicket.text = "ОБРОБКА..."

            CoroutineScope(Dispatchers.IO).launch {
                try {
                    val resultTrain = RetrofitClient.apiService.updateTrain(train.id!!,
updatedTrain)

                    withContext(Dispatchers.Main) {
                        val index = allTrains.indexOfFirst { it.id == train.id }
                        if (index != -1) {
                            val mutableList = allTrains.toMutableList()
                            mutableList[index] = resultTrain
                            allTrains = mutableList

                            val dep = etDeparture.text.toString().trim()
                            val arr = etArrival.text.toString().trim()

                            if (dep.isNotEmpty() && arr.isNotEmpty() &&
selectedSearchDate.isNotEmpty() && !isAdminLoggedIn) {
                                val filteredList = allTrains.filter { t ->
                                    t.startStation.equals(dep, ignoreCase = true) &&
                                            t.endStation.equals(arr, ignoreCase = true) &&
                                            t.departureDate == selectedSearchDate
                                }
                                trainAdapter.updateData(filteredList)
                            } else {
                                trainAdapter.updateData(allTrains)
                            }
                        }

                        Toast.makeText(this@MainActivity, "✅ Оплата успішна!\nВагон
$selectedWagonNum, Місце $seatNum", Toast.LENGTH_LONG).show()
                        dialog.dismiss()
                    }
                } catch (e: Exception) {
                    withContext(Dispatchers.Main) {
                        Toast.makeText(this@MainActivity, "Помилка зв'язку з сервером!",
Toast.LENGTH_SHORT).show()
                        btnBuyTicket.isEnabled = true
                        btnBuyTicket.text = "ОПЛАТИТИ ${train.price} ₴"
                    }
                }
            }
        }
    }

    private fun showAdminLoginDialog() {
        val layout = android.widget.LinearLayout(this).apply {
            orientation = android.widget.LinearLayout.VERTICAL
            setPadding(50, 40, 50, 10)
        }

        val loginInput = EditText(this).apply {
            hint = "Логін"
            inputType = android.text.InputType.TYPE_CLASS_TEXT
        }
        val passwordInput = EditText(this).apply {
            hint = "Пароль"
            inputType = android.text.InputType.TYPE_CLASS_TEXT or
android.text.InputType.TYPE_TEXT_VARIATION_PASSWORD
        }
```

```
        layout.addView(loginInput)
        layout.addView(passwordInput)

        androidx.appcompat.app.AlertDialog.Builder(this)
            .setTitle("Авторизація персоналу")
            .setView(layout)
            .setPositiveButton("Увійти") { dialog, _ ->
                if (loginInput.text.toString() == "admin" && passwordInput.text.toString() ==
"admin") {
                    isAdminLoggedIn = true
                    btnAdminLogin.text = "ВИЙТИ З РЕЖИМУ АДМІНА"
                    btnAdminLogin.setTextColor(android.graphics.Color.parseColor("#D32F2F"))
                    etDeparture.text.clear()
                    etArrival.text.clear()
                    btnSelectDate.text = "ОБРАТИ ДАТУ"
                    selectedSearchDate = ""
                    trainAdapter.updateData(allTrains)
                    trainAdapter.isStaffMode = true
                    trainAdapter.notifyDataSetChanged()
                    fabAddRoute.visibility = View.VISIBLE
                    Toast.makeText(this, "Режим адміна: керування маршрутами активовано",
Toast.LENGTH_LONG).show()
                } else {
                    Toast.makeText(this, "Помилка: Невірний логін або пароль",
Toast.LENGTH_SHORT).show()
                }
                dialog.dismiss()
            }
            .setNegativeButton("Скасувати") { dialog, _ ->
                dialog.dismiss()
            }
            .show()
    }

    private fun deleteRoute(train: TrainRoute) {
        androidx.appcompat.app.AlertDialog.Builder(this)
            .setTitle("Видалення маршруту")
            .setMessage("Ви впевнені, що хочете видалити рейс №${train.trainNumber}?")
            .setPositiveButton("Видалити") { _, _ ->
                CoroutineScope(Dispatchers.IO).launch {
                    try {
                        RetrofitClient.apiService.deleteTrain(train.id!!)

                        withContext(Dispatchers.Main) {
                            allTrains = allTrains.filter { it.id != train.id }
                            trainAdapter.updateData(allTrains)

                            if (etDeparture.text.isNotEmpty() && etArrival.text.isNotEmpty())
{
                                val dep = etDeparture.text.toString().trim()
                                val arr = etArrival.text.toString().trim()
                                val filteredList = allTrains.filter { t ->
                                    t.startStation.equals(dep, ignoreCase = true) &&
                                            t.endStation.equals(arr, ignoreCase = true)
                                }
                                trainAdapter.updateData(filteredList)
                            }

                            Toast.makeText(this@MainActivity, "Рейс назавжди видалено з БД",
Toast.LENGTH_SHORT).show()
                        }
                    } catch (e: Exception) {
                        withContext(Dispatchers.Main) {
                            Toast.makeText(this@MainActivity, "Помилка сервера при видаленні",
Toast.LENGTH_SHORT).show()
                        }
```

```
                }
            }
        }
        .setNegativeButton("Скасувати", null)
        .show()
    }

    private fun showAddRouteDialog() {
        val layout = android.widget.LinearLayout(this).apply {
            orientation = android.widget.LinearLayout.VERTICAL
            setPadding(50, 40, 50, 10)
        }

        val numInput = EditText(this).apply { hint = "Номер потяга" }
        val fromInput = EditText(this).apply { hint = "Звідки" }
        val toInput = EditText(this).apply { hint = "Куди" }
        val timeInput = EditText(this).apply { hint = "Час (напр. 14:00)" }

        val dateInput = EditText(this).apply {
            hint = "Оберіть дату"
            isFocusable = false
            isClickable = true
        }

        var finalDate = ""

        dateInput.setOnClickListener {
            val calendar = java.util.Calendar.getInstance()
            android.app.DatePickerDialog(this, { _, year, month, day ->
                finalDate = String.format("%02d.%02d.%04d", day, month + 1, year)
                dateInput.setText(finalDate)
            }, calendar.get(java.util.Calendar.YEAR), calendar.get(java.util.Calendar.MONTH),
calendar.get(java.util.Calendar.DAY_OF_MONTH)).show()
        }

        layout.addView(numInput)
        layout.addView(fromInput)
        layout.addView(toInput)
        layout.addView(timeInput)
        layout.addView(dateInput)

        androidx.appcompat.app.AlertDialog.Builder(this)
            .setTitle("Додати новий маршрут")
            .setView(layout)
            .setPositiveButton("Зберегти") { dialog, _ ->
                val newRoute = TrainRoute(
                    tripId = (10..999).random(),
                    trainNumber = numInput.text.toString(),
                    startStation = fromInput.text.toString(),
                    endStation = toInput.text.toString(),
                    departureDate = finalDate,
                    departureTime = timeInput.text.toString(),
                    price = 500.0,
                    trainType = "Пасажирський",
                    carriageCount = 10,
                    status = "OpenForBooking",
                    bookedSeats = ""
                )

                CoroutineScope(Dispatchers.IO).launch {
                    try {
                        val createdTrain = RetrofitClient.apiService.addTrain(newRoute)

                        withContext(Dispatchers.Main) {
                            val updatedList = allTrains.toMutableList()
                            updatedList.add(createdTrain)
```

```
                            allTrains = updatedList
                            trainAdapter.updateData(allTrains)
                            Toast.makeText(this@MainActivity, "Рейс успішно збережено на
сервері!", Toast.LENGTH_SHORT).show()
                        }
                    } catch (e: Exception) {
                        withContext(Dispatchers.Main) {
                            Toast.makeText(this@MainActivity, "Помилка збереження на сервер",
Toast.LENGTH_SHORT).show()
                        }
                    }
                }
                dialog.dismiss()
            }
            .setNegativeButton("Скасувати") { dialog, _ -> dialog.dismiss() }
            .show()
    }

    private fun showEditRouteDialog(train: TrainRoute) {
        val layout = android.widget.LinearLayout(this).apply {
            orientation = android.widget.LinearLayout.VERTICAL
            setPadding(50, 40, 50, 10)
        }

        val numInput = EditText(this).apply { setText(train.trainNumber) }
        val fromInput = EditText(this).apply { setText(train.startStation) }
        val toInput = EditText(this).apply { setText(train.endStation) }
        val timeInput = EditText(this).apply { setText(train.departureTime) }

        val dateInput = EditText(this).apply {
            setText(train.departureDate)
            isFocusable = false
            isClickable = true
        }

        var finalDate = train.departureDate

        dateInput.setOnClickListener {
            val calendar = java.util.Calendar.getInstance()
            val (day, month, year) = parseDate(finalDate ?: "")
            android.app.DatePickerDialog(this, { _, selectedYear, selectedMonth, selectedDay
->
                finalDate = String.format("%02d.%02d.%04d", selectedDay, selectedMonth + 1,
selectedYear)
                dateInput.setText(finalDate)
            }, year, month, day).show()
        }

        val priceInput = EditText(this).apply { setText(train.price.toString()) }

        layout.addView(numInput)
        layout.addView(fromInput)
        layout.addView(toInput)
        layout.addView(timeInput)
        layout.addView(dateInput)
        layout.addView(priceInput)

        androidx.appcompat.app.AlertDialog.Builder(this)
            .setTitle("Редагувати маршрут")
            .setView(layout)
            .setPositiveButton("Оновити") { dialog, _ ->
                val updatedRoute = train.copy(
                    trainNumber = numInput.text.toString(),
                    startStation = fromInput.text.toString(),
                    endStation = toInput.text.toString(),
                    departureTime = timeInput.text.toString(),
```

```
                    departureDate = finalDate,
                    price = priceInput.text.toString().toDoubleOrNull() ?: train.price
                )

                CoroutineScope(Dispatchers.IO).launch {
                    try {
                        val resultTrain = RetrofitClient.apiService.updateTrain(train.id!!,
updatedRoute)

                        withContext(Dispatchers.Main) {
                            val index = allTrains.indexOfFirst { it.id == train.id }
                            if (index != -1) {
                                val mutableList = allTrains.toMutableList()
                                mutableList[index] = resultTrain
                                allTrains = mutableList
                                trainAdapter.updateData(allTrains)
                                Toast.makeText(this@MainActivity, "Рейс успішно оновлено!",
Toast.LENGTH_SHORT).show()
                            }
                        }
                    } catch (e: Exception) {
                        withContext(Dispatchers.Main) {
                            Toast.makeText(this@MainActivity, "Помилка оновлення на сервері",
Toast.LENGTH_SHORT).show()
                        }
                    }
                }
                dialog.dismiss()
            }
            .setNegativeButton("Скасувати") { dialog, _ -> dialog.dismiss() }
            .show()
    }

    private fun parseDate(dateString: String): Triple<Int, Int, Int> {
        val parts = dateString.split(".")
        val day = parts[0].toInt()
        val month = parts[1].toInt() - 1
        val year = parts[2].toInt()
        return Triple(day, month, year)
    }
}
```

SeatAdapter.kt Лістинг:

```
package com.example.trainschedule

import android.graphics.Color
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.TextView
import androidx.recyclerview.widget.RecyclerView
import com.google.android.material.card.MaterialCardView

enum class SeatStatus { FREE, OCCUPIED, SELECTED }
data class Seat(val number: Int, var status: SeatStatus)

class SeatAdapter(
    private var seats: List<Seat>,
    private val onSeatSelected: (Seat?) -> Unit
) : RecyclerView.Adapter<SeatAdapter.SeatViewHolder>() {

    private var selectedSeatIndex: Int = -1
```

```
    class SeatViewHolder(view: View) : RecyclerView.ViewHolder(view) {
        val cardSeat: MaterialCardView = view.findViewById(R.id.cardSeat)
        val tvSeatNumber: TextView = view.findViewById(R.id.tvSeatNumber)
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): SeatViewHolder {
        val view = LayoutInflater.from(parent.context).inflate(R.layout.item_seat, parent,
false)
        return SeatViewHolder(view)
    }

    override fun onBindViewHolder(holder: SeatViewHolder, position: Int) {
        val seat = seats[position]
        holder.tvSeatNumber.text = seat.number.toString()

        when (seat.status) {
            SeatStatus.FREE -> {
                holder.cardSeat.setCardBackgroundColor(Color.WHITE)
                holder.cardSeat.strokeColor = Color.parseColor("#E5E5EA")
                holder.tvSeatNumber.setTextColor(Color.parseColor("#1C1C1E"))
            }
            SeatStatus.OCCUPIED -> {
                holder.cardSeat.setCardBackgroundColor(Color.parseColor("#F2F2F7"))
                holder.cardSeat.strokeColor = Color.TRANSPARENT
                holder.tvSeatNumber.setTextColor(Color.parseColor("#D1D1D6"))
            }
            SeatStatus.SELECTED -> {
                holder.cardSeat.setCardBackgroundColor(Color.parseColor("#5E35B1"))
                holder.cardSeat.strokeColor = Color.parseColor("#5E35B1")
                holder.tvSeatNumber.setTextColor(Color.WHITE)
            }
        }

        holder.itemView.setOnClickListener {
            if (seat.status == SeatStatus.OCCUPIED) return@setOnClickListener

            if (seat.status == SeatStatus.SELECTED) {
                seat.status = SeatStatus.FREE
                selectedSeatIndex = -1
                onSeatSelected(null)
            } else {
                if (selectedSeatIndex != -1) {
                    seats[selectedSeatIndex].status = SeatStatus.FREE
                }
                seat.status = SeatStatus.SELECTED
                selectedSeatIndex = position
                onSeatSelected(seat)
            }
            notifyDataSetChanged()
        }
    }

    override fun getItemCount() = seats.size

    fun updateSeats(newSeats: List<Seat>) {
        seats = newSeats
        selectedSeatIndex = -1
        notifyDataSetChanged()
    }
}
```

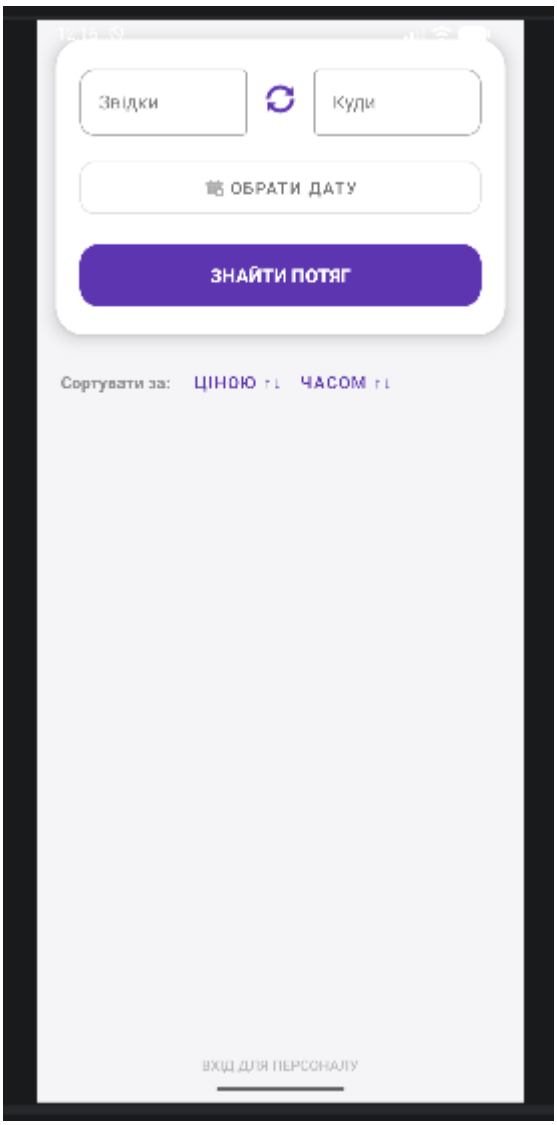

Рисунок 1 – Головний екран пошуку з полями вводу станцій.

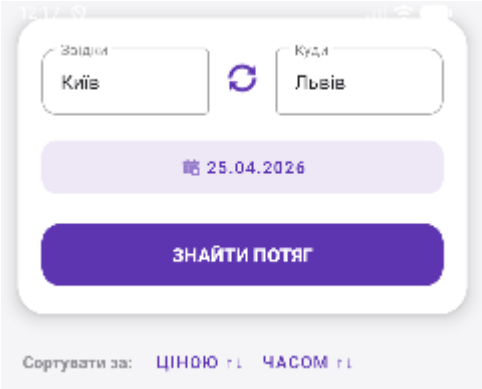

Рисунок 2 – Робота системного компонента DatePickerDialog для вибору дати поїздки та введені станції.

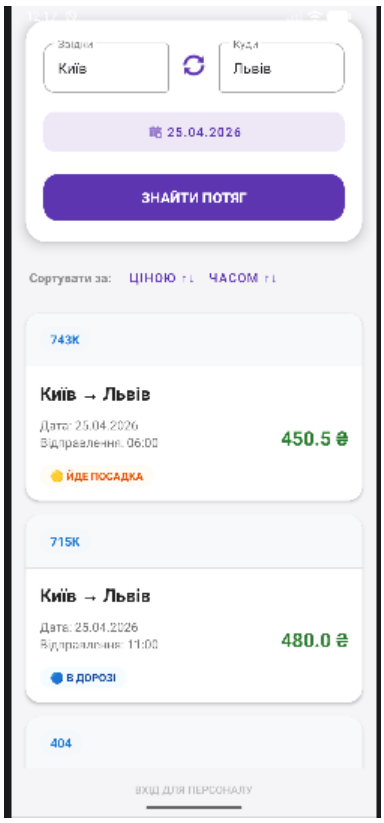

Рисунок 3 – Відображення результатів пошуку (список рейсів реалізовано через RecyclerView).

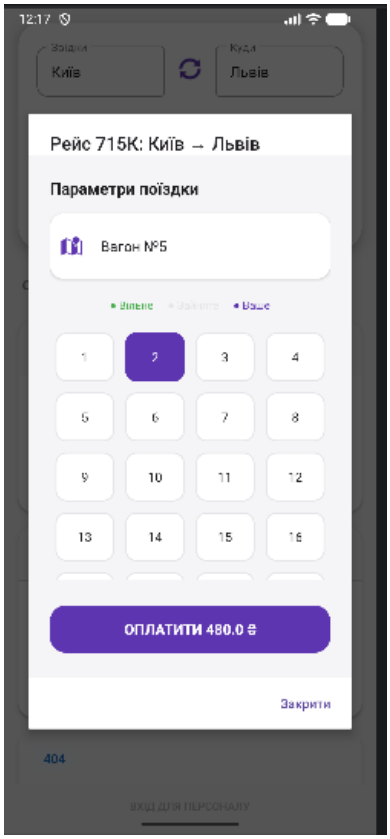

Рисунок 4 – Інтерактивне діалогове вікно вибору вагона та місця із кольоровою індикацією статусів.

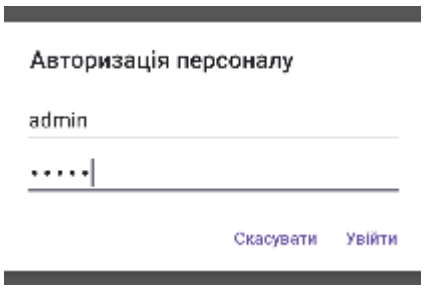

Рисунок 5 – Авторизація адміна

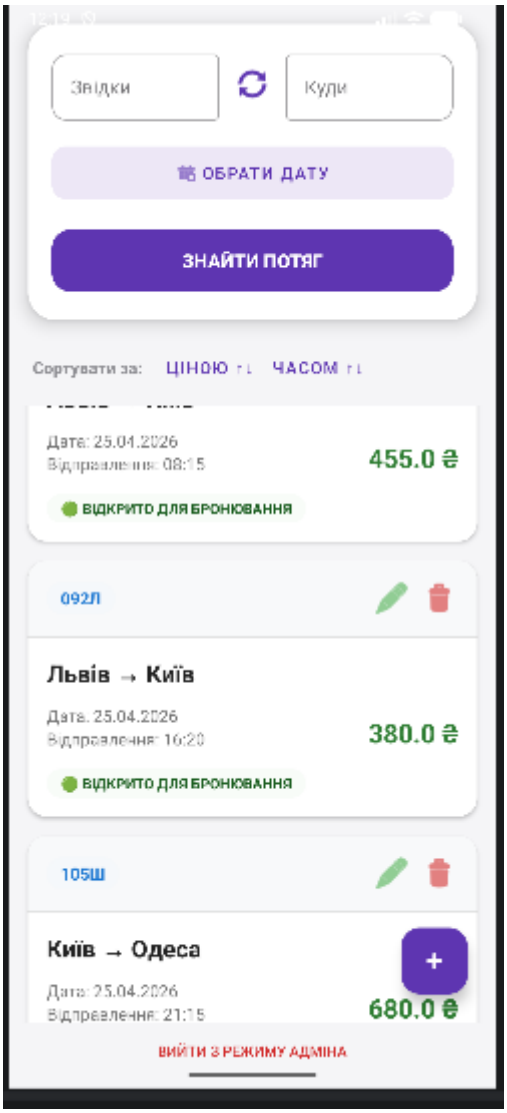

Рисунок 6 – Головний екран від імені адміністратора

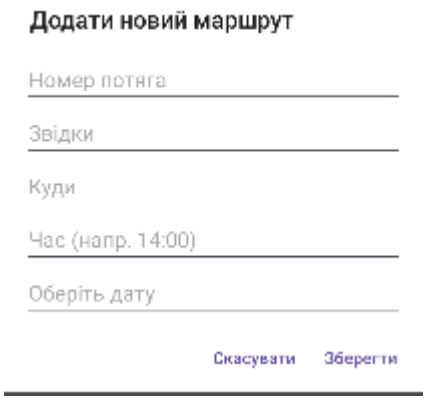

Рисунок 7 – Інтерактивне діалогове вікно для додавання маршруту

Рисунок 8 – Наш створений API

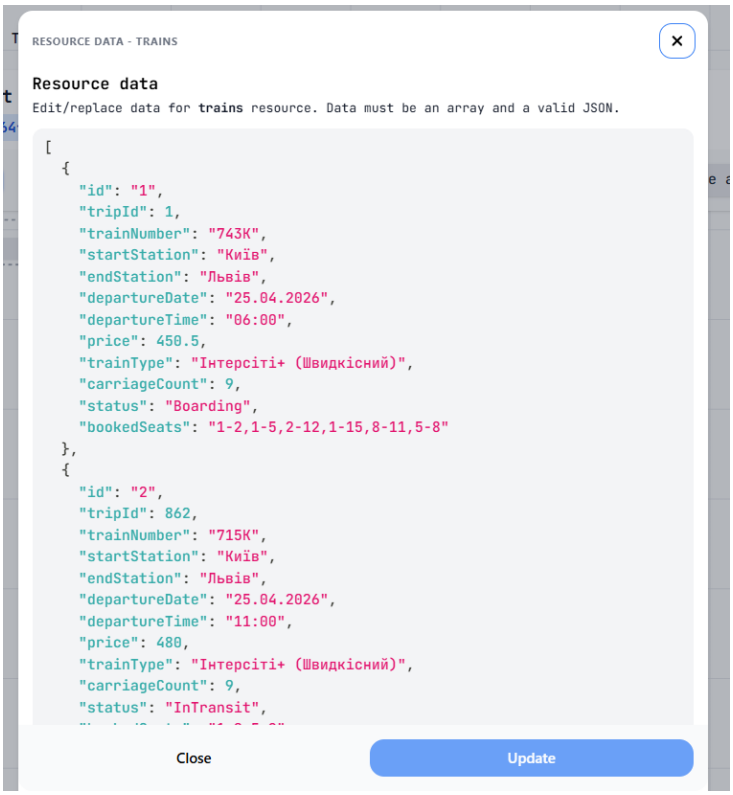

Рисунок 9 – Дані про рейси на сторінці API

4. Висновок

Під час виконання лабораторної роботи було змодельовано розробку комплексного клієнт-серверного застосунку, який може слугувати прототипом для дипломного проєкту. Мною було успішно досліджено та інтегровано технології, що не розглядалися в попередніх роботах, зокрема: роботу з віддаленими базами даних через REST API за допомогою бібліотеки Retrofit та обробку мережевих запитів в асинхронному режимі з використанням Kotlin Coroutines. Реалізовано повноцінний життєвий цикл обробки даних (CRUD): отримання розкладу з сервера, локальна фільтрація (за датою, станціями, ціною, часом), а також модифікація даних на сервері (імітація купівлі квитка з оновленням масиву зайнятих місць, додавання нових маршрутів адміністратором). Графічний інтерфейс побудовано з використанням сучасних компонентів Material Design та оптимізованих списків RecyclerView, що забезпечує високий рівень користувацького досвіду (UX). Мета роботи досягнута в повному обсязі, закладено потужний технологічний базис для подальшого масштабування системи.